| | | | | | |
|---|---|---|---|---|---|
| | **Academic Year:** | **2022/2023** | **Term:** | **Spring 2023** | |
| | **Course Code:** | **ELC4007** | **Course Title:** | **Digital Design** | |

# Cairo University

# Faculty of Engineering

# Electronics and Communications Engineering Department

# 4th Year

Design of a 4-Tap FFE Project

| Student Name | | Section | B.N. | StdSeatNo | StdCode |
|---|---|---|---|---|---|
| 1 | احمد ممدوح احمد ابوضيف | 1 | 23 | 34023 | 9190617 |
| 2 | ايمان زكريا شعبان عرفه | 1 | 40 | 34040 | 9190355 |
| 3 | عمر احمد فهمى احمد | 3 | 10 | 34122 | 9190049 |
| 4 | عمرو ماجد حسن طه الجزار | 3 | 19 | 34131 | 9190632 |

# Table of Contents


Introduction ..... 3

System Modeling (Extra) ..... 3

Hardware Architecture ..... 5

- Synchronous clock domains case ..... 5
- Asynchronous clock domains case ..... 6
- Other Approaches for the Different Clock Domains Case (Extra) ..... 7
  - Using an Asynchronous FIFO ..... 7
  - Using a Handshaking Protocol ..... 7
  - Using a Delay Locked Loop (DLL) ..... 8

RTL Design and Testbench (Extra) ..... 9

- Testbench ..... 9
- RTL ..... 9

FPGA Prototyping (Extra) ..... 10

- Board Information ..... 10
- Test Environment ..... 10
  - *clock_wiz* ..... 11
  - *rst_sync* ..... 11
  - bit_sync ..... 11
  - ROMs ..... 11
  - ffe_top ..... 11
  - control_unit ..... 11
- Results ..... 13

# Introduction

1. Why do we need equalization?
    - In a digital communication system, an equalizer is used to compensate for distortions and attenuations that occur when a signal is transmitted through a channel. Equalization at Tx and Rx is needed to counter the effects of the channel.
2. What is a FFE?
    - A Feed-Forward Equalizer (FFE) is a linear equalizer that applies a linear filter to the transmitted signal to pre-distort it before sending it through the channel, and then uses a linear filter at the receiver to estimate the channel's impulse response and apply an inverse filter to cancel out the distortion. The FFE typically consists of a series of taps that apply different weights to the transmitted signal. Each tap represents a specific time delay of the channel's impulse response. The weights applied by the taps are determined by the characteristics of the channel.
3. What is the FFE's equation?
    - It is nothing but an FIR filter with 4 taps whose equation is given by:

$$Y_i = \sum_{j=0}^{3} h_j \, D_{i-j}$$

# System Modeling (Extra)

We have modeled the system on MATLAB so that we have a golden (reference) model. The model consists of three main scripts:

1. Floating-point script: this is a script that describes the expected behavior of the system on a high level using floating-point operations. That is, it uses any built-in functions in MATLAB and uses full precision to model the behavior, it does not care about neither fixed-point size nor digital design aspects (area, power, timing, etc.). Regarding the input to the script, we have generated OFDM symbols to use as input samples for the design. The section that describes the FFE equation in the model is shown in figure 1:

```
%% FFE Equalizer
currentSamples = zeros(4, 1);
for i = 1 : length(OFDMSymbols)
    currentSamples = [OFDMSymbols(i); currentSamples(1:3)];
    ffe = currentSamples(1) * h(1) + currentSamples(2) * h(2) + currentSamples(3) * h(3) + currentSamples(4) * h(4);
end
```

*Figure 1. FFE Equalizer Section in the Floating-Point Script*

2. Fixed-point script pre-RTL: this is a script that describes exactly what would happen in the RTL. That is, it exactly models each operation (e.g. addition, multiplication, etc.) that will occur in the RTL. For example, if there are adders/multipliers whose outputs in the RTL will be quantized (i.e. some bits will be truncated), this should be modeled in this script. All variables are defined as fixed-point variables with a parameterized integer and fractional parts bits. This can be done using the *fi* object on MATLAB that offers us the opportunity to model a fixed-point environment, which is the RTL environment, in a floating-point environment, which is MATLAB. This is shown in figure 2. Notice the difference between the section below and the one we saw in the floating-point script. Here, we exactly model each multiplication and addition operation as well as perform quantization after each operation (thanks to the *fi* object), which imitates what will actually happen on hardware.

```
%% FFE Equalizer
currentSamples = zeros(4, 1);
currentSamples_fixed = fi(currentSamples, 1, Integer_Part+Fractional_Part, Fractional_Part, F);
for i = 1 : length(OFDMSymbols)
    currentSamples_fixed = [OFDMSymbols_fixed(i); currentSamples_fixed(1:3)];
    x1 = currentSamples_fixed(1) * h_fixed(1);
    x2 = currentSamples_fixed(2) * h_fixed(2);
    x3 = currentSamples_fixed(3) * h_fixed(3);
    x4 = currentSamples_fixed(4) * h_fixed(4);
    x5 = x1 + x2;
    x6 = x3 + x4;
    ffe = x5 + x6;
end
```

*Figure 2. FFE Equalizer Section in the Fixed-Point Script*

3. Fixed-point script post-RTL: similar to the pre-RTL script, this script also defines all variables as fixed-point variables. The difference here is that this script is not used to model the behavior of the design. Instead, it is used to generate text files that contains all the inputs that to be passed to the DUT, which are mainly the input samples and FIR filter coefficients here, as well as a text file that contains the expected outputs from the DUT so that we later compare the output of the RTL with this text file and check whether our RTL is correct or not. A glimpse of the source code generating the input text files for the DUT is shown below:

```
%% Generate Text Files Containing The Inputs to be Passed to the DUT
OFDMSymbols_fixed_bin = OFDMSymbols_fixed.bin;
h_fixed_bin = h_fixed.bin;
fileID1 = fopen('Input_Samples.txt', 'w');
for i = 1 : size(OFDMSymbols_fixed, 1)
    if i == size(OFDMSymbols_fixed,1)
      fprintf(fileID1, '%s', OFDMSymbols_fixed_bin(i, :));
    else
      fprintf(fileID1, '%s\n', OFDMSymbols_fixed_bin(i, :));
    end
end
```

*Figure 3. DUT Input Text Files Generation*

Another section showing the reading of the text files generated by the RTL is shown in figure 4. Notice that the generated text files from the RTL are in binary, so we convert them to decimal representation for convenient visualization on MATLAB.

```
%% Read the Text Files Generated by the RTL to Check the Output
fileID1 = fopen('../Questa_Project/ffe_outputs.txt','r');
content1 = textscan(fileID1, '%s', 'Delimiter', '\n');
rtlOutBinFile = char(string(content1{1}));
rtlOutDecFile = q2dec(rtlOutBinFile, Integer_Part-1, Fractional_Part, 'bin');
```

*Figure 4. Reading DUT Output Text Files*

## Hardware Architecture

The hardware architecture is composed of two main parts: controller and datapath. The datapath mainly contains the operations required by the FFE, while the controller controls the flow of data in the datapath. The controller does so through receiving status signals from the datapath and sending control signals to the datapath.

The input data coming from the transmitter side can be synchronous or asynchronous to our clock domain. We have handled both cases (i.e. designed an architecture for each case) and we will discuss both of them in the next part.

### Synchronous clock domains case

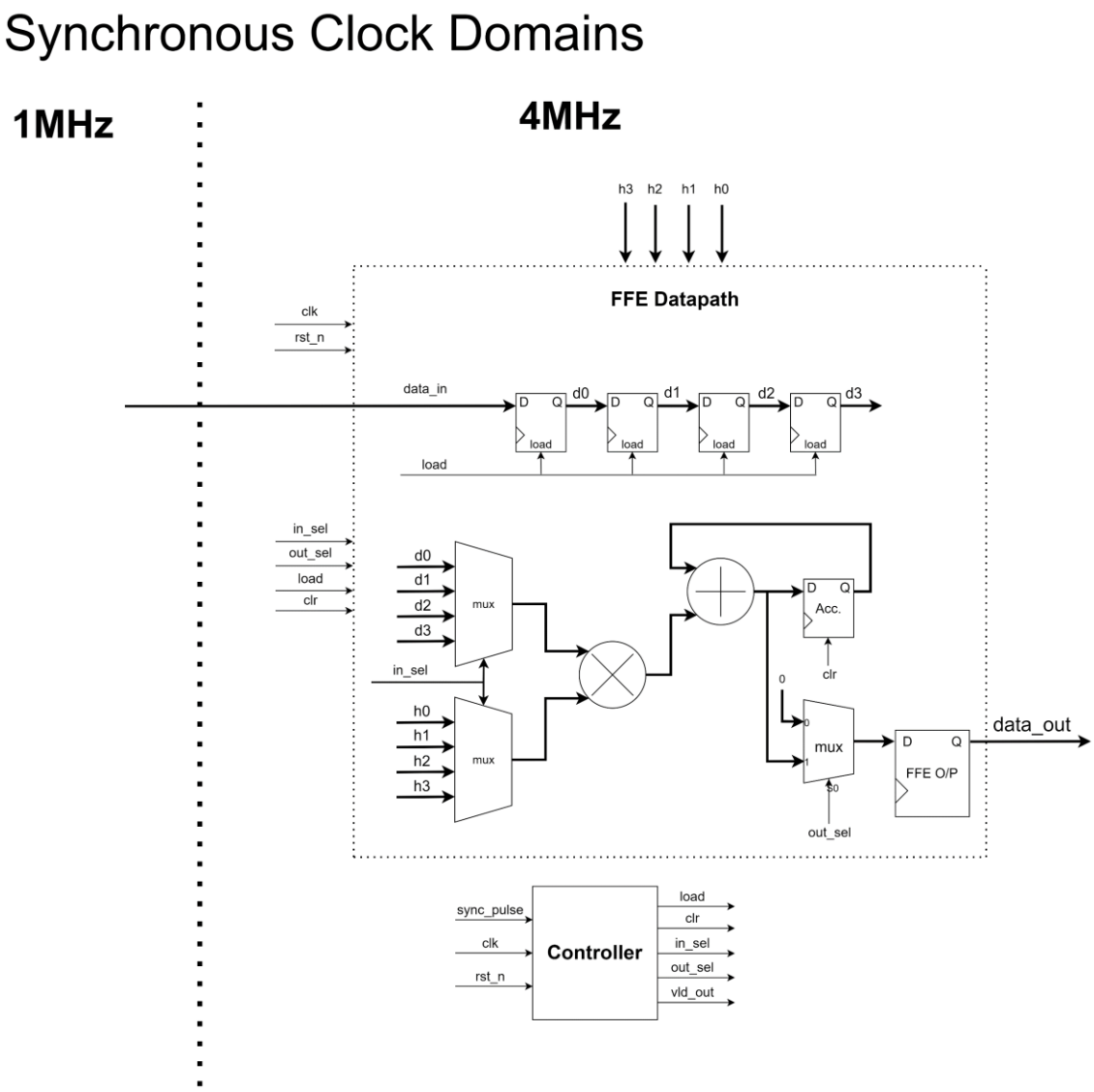


*Figure 5. Synchronous clock domains FFE architecture*

Shown in Figure 5 is the architecture for the FFE in case of synchronous domains. Since the two clock domains are synchronous, we can safely apply the input data *data_in* coming from the transmitter side (i.e. 1 MHz) directly to registers operating with the FFE's clock (4 MHz) and rely on the load signal asserted by the controller to latch the new data on the 4MHz required clock cycle. The load signal is asserted every four 4MHz clock cycles. This data is shifted each 1us to allow new input data receiving and new calculations based on it.

Time sharing is used to allow usage of only one multiplier and one adder. This is achieved through using a 4x1 Multiplexer whose select signal is updated each clock cycle of the 4MHz clock. This select signal is driven by the controller.

Notice that the filter's coefficients are not hard coded to the values in the project statement, they are passed as inputs to the filter. This way, we can change the filter's coefficients as we want on MATLAB and simply pass them to the RTL. This is the actual case for most of the filters that their tap coefficients are updated from time to time based on the channel conditions.

The result of multiplication and addition is accumulated to allow reusage of the resources yet producing reliable results. The data stored in the accumulator is cleared each 1 us for the sake of new data calculations.

The controller asserts the output select and valid signal on the required clock cycle to indicate output data reliability to the top module.

## Asynchronous clock domains case

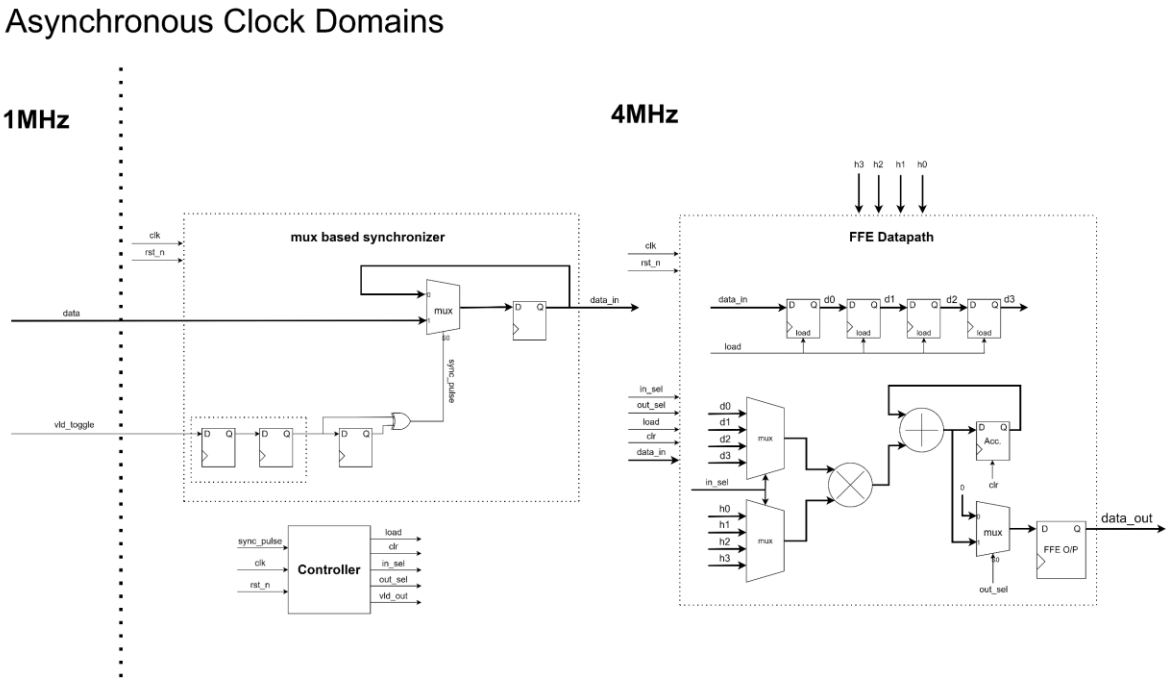


*Figure 6. Asynchronous clock domains FFE architecture.*

In the asynchronous clock domains case, we can use the same architecture explained in the synchronous case, but with two main differences. The first difference is that we are going to need a bus synchronizer to ensure latching input data safely (i.e. synchronous to the 4-MHz domain). The second difference is based on the first one, as this synchronizer will need the transmitter to send a toggling signal when the data transmitted from his side changes. since the transmitter sends a continuous stream of data, we will need this signal to toggle each cycle.

To explain this, we need to focus on the relation between the two asynchronous clock domains frequencies. As we see, the receiving clock frequency is four times faster than that of the transmitting clock.

This can allow the usage of the simple multiplexer-based synchronizer, which is based on the synchronization of the signal which toggle whenever a new data is put by the transmitter on the data bus. The clock frequencies relation can allow synchronization of this signal through a double-

flip flop synchronizer to produce only an 0.25-us pulse when the new bus data is 100% ensured to be stable. This will allow latching this data safely inside the 4 MHz clock domain.

This latched data can be considered the new synchronous input which is latched inside the shift register through the signal asserted by the controller. And the operation continues the same as the previously explained synchronous clock domains case.

## Other Approaches for the Different Clock Domains Case (Extra)

### Using an Asynchronous FIFO

Another more complex yet reliable approach that can be used for synchronization is using an asynchronous FIFO (First In First Out). This will need additional signals for the transmitter to handle which are write enable which maybe tied to logic high in case that the FIFO is not going to be full under any circumstances. This approach is shown in the next figure.

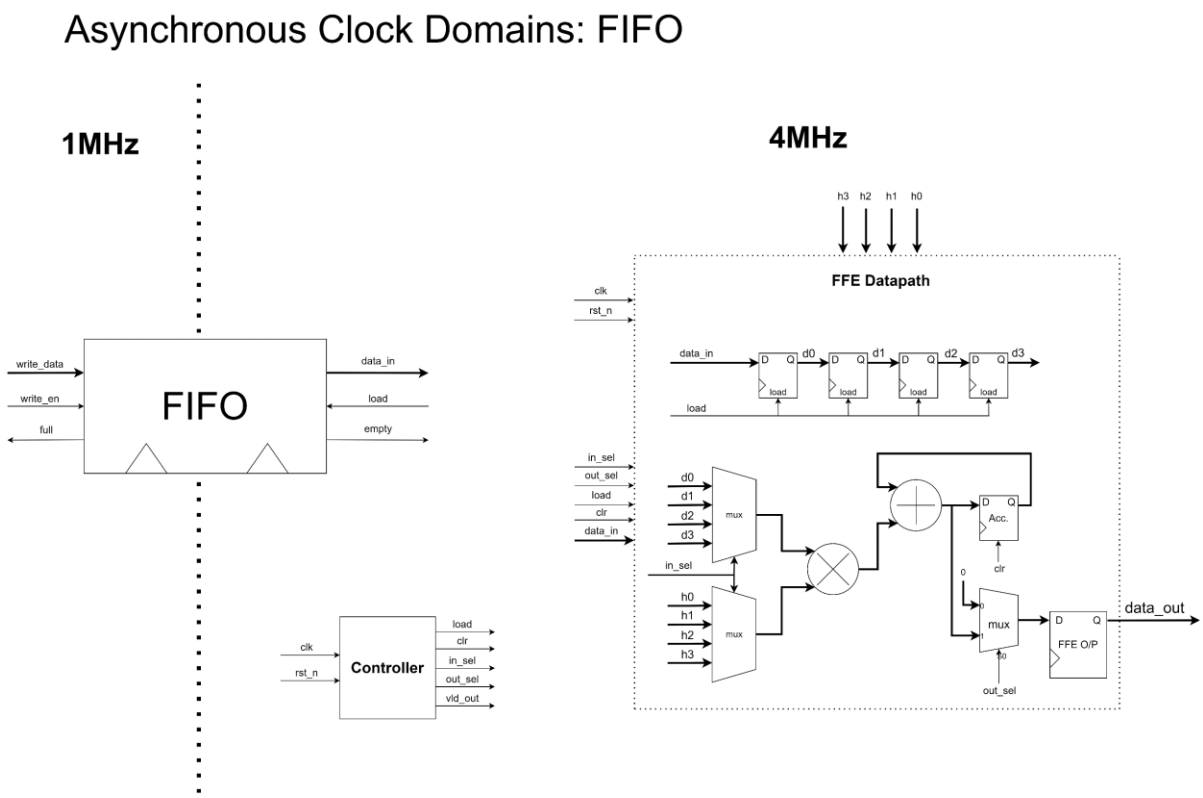


*Figure 7. FFE Architecture Using an Asynchronous FIFO Approach for Synchronization*

***Disadvantage of this approach compared to our approach***: using an asynchronous FIFO in our case is inefficient because the transmitter is sending with a data rate lower than the receiving rate of the receiver. Usually, an asynchronous FIFO is used in case of bursty data that is being sent from a faster domain to a slower domain. But in case of transmitting from a slower domain to a faster domain, we don't really need to overwhelm ourselves with the many control signals required by the FIFO.

### Using a Handshaking Protocol

Another more complex approach that can be used for synchronization is using handshaking circuitry. The transmitter would require sending a request signal while having the data stable on the data bus and wait for an acknowledgment. Once the acknowledgment signal has been received, the transmitter can send another data. This approach is shown in figure 8.

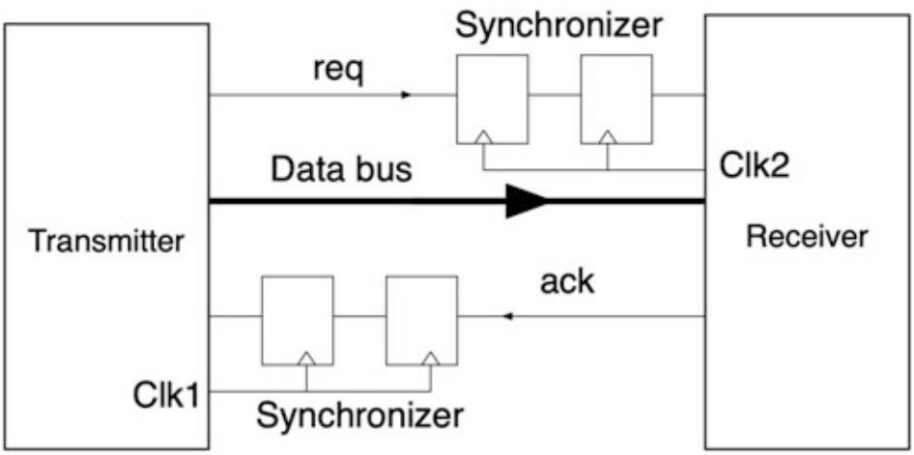


*Figure 8. Handshaking Circuitry Approach for Synchronization*

***Disadvantage of this approach compared to our approach***: using handshaking circuitry in our case is the most inefficient approach because too many clock cycles are wasted for just handing in 1 sample from the transmitter to the receiver. In case of using this FFE filter in a fast data-rate communication system (e.g. 5G transceiver), this approach would deteriorates the data rate.

## Using a Delay Locked Loop (DLL)

Since the two domains have clock frequencies that are multiple of each other (i.e. $f_{Rx} = 4f_{Tx}$), our only problem for synchronization is that the clocks do not have a constant phase relationship. Thus, we can make use of this feature and use a DLL that can change the phase of one of the two clock signals so that they become synchronous to each other. The main component of a DLL is a delay chain that is used to change the phase of the input clock signal by passing it through a number of delay elements. This is shown in figure 9.

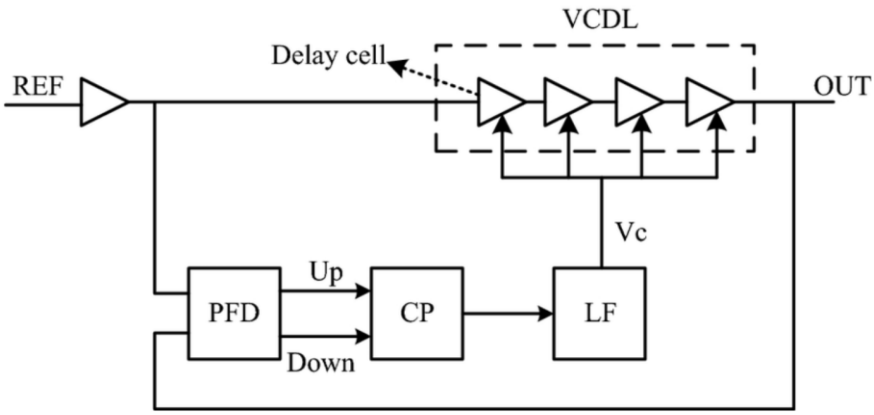


*Figure 9. DLL Approach for Synchronization*

***Disadvantage of this approach compared to our approach***: DLLs can be quite complex to design and implement compared to other synchronization techniques. They require careful consideration of various parameters, such as delay line lengths, clock edges, and control signals. Also, they can be sensitive to jitter and noise in the input clock signals. If the input clock have significant levels of jitter or noise, the DLL's performance may degrade, leading to potential synchronization issues or inaccuracies in the output. Finally, implementing a DLL requires additional circuitry, such as delay lines, phase detectors, and control logic. This increases the overall circuit area and cost, which may not be desirable in cost-sensitive or space-constrained designs.

# RTL Design and Testbench (Extra)

## Testbench

The testbench reads the input stimulus generated from the MATLAB model through text files as discussed before. Then, the TB drives the DUT input ports, which includes both the input data and the FFE coefficients.

The testbench verifies the intended behavior of our synchronization scheme in case we have asynchronous clocks through generating two clocks of misaligned phases. The 1MHz clock is used to apply the inputs to the DUT and the 4 MHz clock is used by the FFE itself.

After applying the inputs to the DUT, the TB polls on the valid signal indicating that the DUT output is valid and writes those outputs to a text file in order to be compared with the expected outputs generated from the MATLAB model.

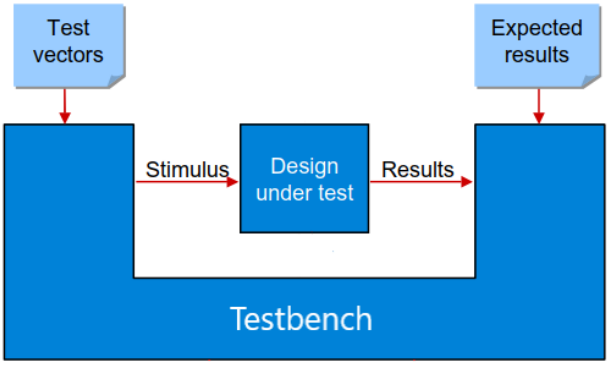


*Figure 9 Testbench Structure*

## RTL

After instantiating the DUT in the testbench and driving its inputs using asynchronous clocks, the synchronizer behavior is as shown in Figure 10.

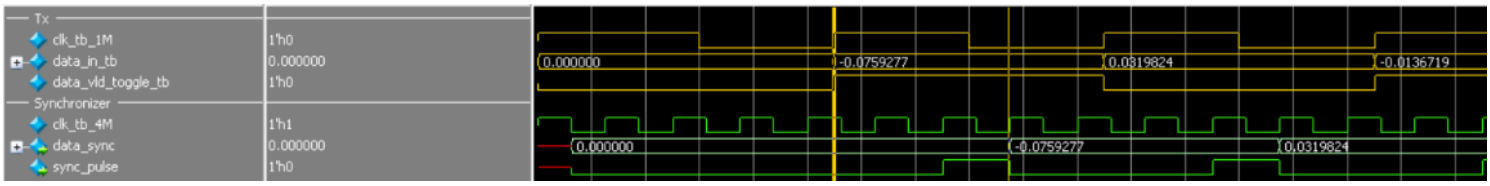


*Figure 10. The Synchronizer Waveform*

The data is applied to the synchronizer input using the 1MHz clock, where the phases of the 1 MHz clock and the 4 MHz clock are misaligned as shown in the figure. The synchronizer generates the data at the edges of the 4 MHz preceded by a synchronized single-cycle pulse.

The FFE then reads the synchronized data and generates the output every 4 cycles of the 4 MHz as intended at an effective rate of 1 MHz, as shown in Figure 11.

*Figure 11. The FFE waveform*

# FPGA Prototyping (Extra)

We have requested an access to an FPGA board in the department's lab (Hammam's lab) to prototype our project on to validate its reliability on hardware (thanks to Dr. Serag).

## Board Information

For FPGA prototyping, the product family we used from Xilinx is ***ZYNQ Ultrascale+ MPSoC***. Any product from this family contains an FPGA device that consists of two parts:

1. Processing System (PS): a processing subsystem that includes the ARM Cortex-A53 and Cortex-R5 processors as well as other hard IP blocks such as memory controllers, Ethernet MACs, USB controllers, etc.
2. Programmable Logic (PL): the reconfigurable logic fabric that we are usually interested in that contains logic elements, BRAMs, DSP slices, etc.

The PS and PL are connected via a high-band width low-latency AMBA AXI interface, which allows for efficient communication between the two domains inside the FPGA device. So, notice that the FPGA device integrates both the PS and PL, so when we refer to an FPGA device, we are referring to both the PS and the PL, not just the PL.

The previous applies for all the FPGA devices from the *ZYNQ Ultrascale+ MPSoC* family. An evaluation board/kit is a board that contains an FPGA device in addition to other capabilities (e.g. USB, ethernet, SD card, expansion connectors, DDR memories, etc.). The board we will be using is the ***ZCU104*** evaluation board (The user guide of the board can be found here). The FPGA device used on this board is the ***XCZU7EV*** device.

## Test Environment

The method we will be using depends on using ROMs for storing the inputs that will be passed to the DUT as well as a ROM for storing the reference output that will be compared to the DUT's output. The flow is as follows:

1. Write the input test vectors for your design in a ROM.
2. Write the expected output vectors from your design in a ROM.
3. Design a control unit that manages reading data from both ROMs.
4. Use an Integrated Logic Analyzer (ILA) to monitor the outputs of your design and make sure they match the ones stored in the output ROMs.

The test environment is shown in the figure below:

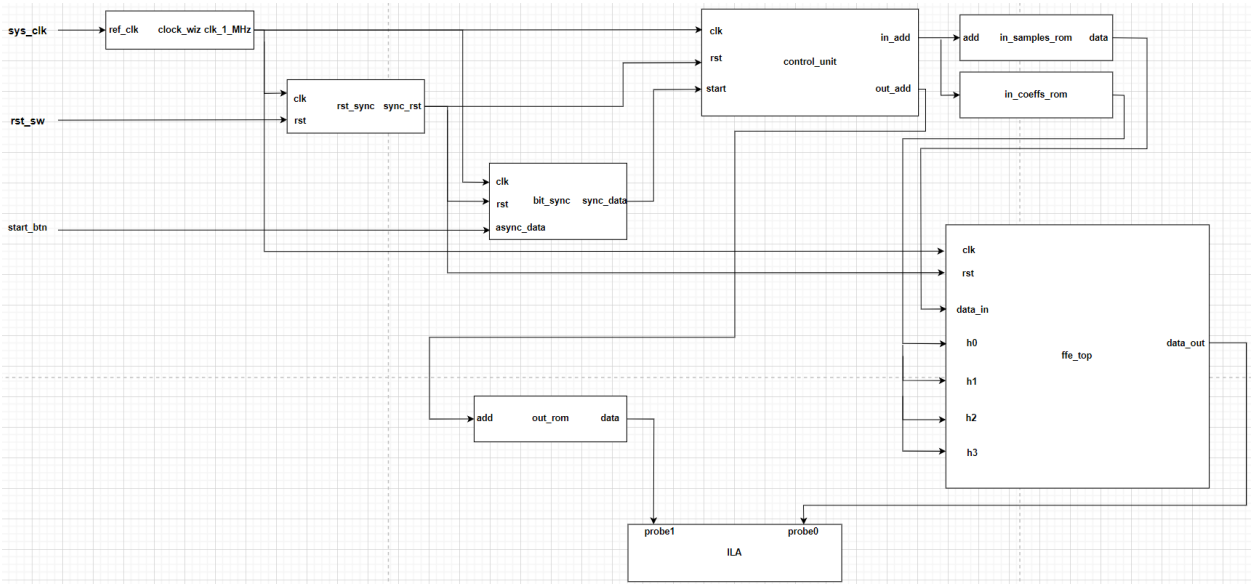


*Figure 12. FFE Test Environment on FPGA*

## *clock_wiz*

- A clocking wizard, named *clock_wiz,* is used to generate the clock required. The input clock *sys_clk* is an on-board clock coming from a clock oscillator found on the board. It is coming with a frequency of *33.33 MHz.* However, we need a *1-MHz* clock, so this is the function of the clocking wizard, which is nothing but a PLL that takes in a clock signal and generates an output clock with the required frequency (i.e. frequency synthesis).

## *rst_sync*

- We are using an on-board switch for resetting, so a reset synchronizer, named *rst_sync,* is needed to synchronize this switch to our clock domain since this switch is an asynchronous signal.

## bit_sync

- We are using an on-board pushbutton for starting the operation, so a bit synchronizer, named *bit_sync,* is needed to synchronize this switch to our clock domain since this switch is an asynchronous signal.

## ROMs

- Two ROMs are used to store the inputs (samples and coefficients) to be passed to our design. Another ROM is used to store the reference/expected outputs from our design.

## ffe_top

- This is the Design Under Test (DUT).

## control_unit

- It is responsible for generating addresses for the input and output ROMs.

- These addresses are generated by using two counters:

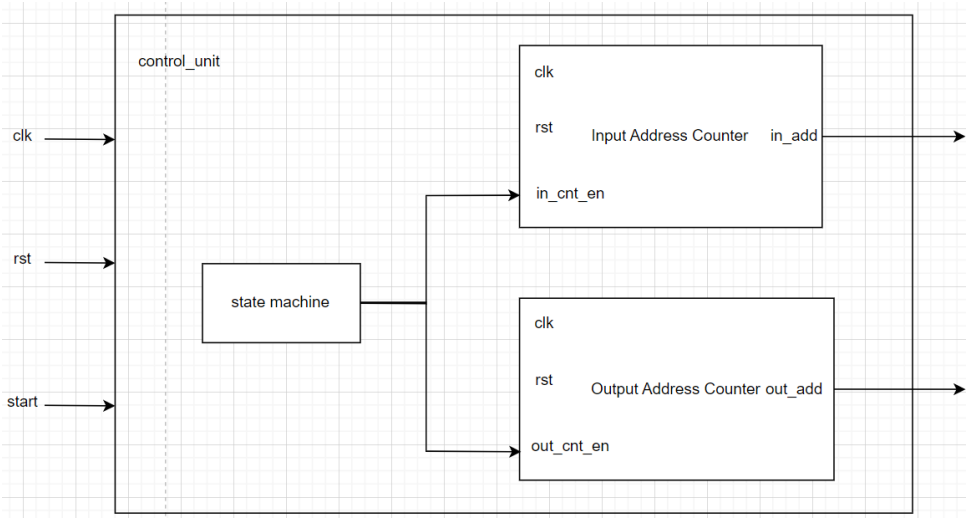


*Figure 14. Control Unit*

and the state machine state diagram is as shown below:

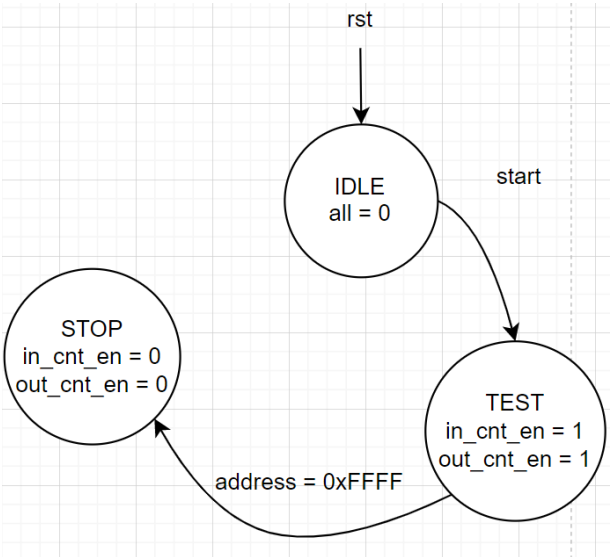


*Figure 15. State Diagram*

Why do we go to a *STOP* state rather than returning to the *IDLE* state?

Because the *IDLE* state is triggered by the *start* signal. At the same time, the *start* signal is initiated using a pushbutton. Thus, we might press the start button to initiate the circuit, but the circuit is too fast that it went through all states and returned back to the *idle* state while we still not have deactivated the *start* button (i.e. haven't removed our finger off the button). So, the safest way is to go to an *end* state, and if you would like to retrigger the circuit from the beginning, simply *reset* the circuit to return to the *IDLE* state.

## Results

Since we are using an FPGA from Xilinx, we used **Vivado** to synthesize, implement, and generate a bitstream file for our test environment.

Figure 16 shows a part of the results seen live on the FPGA thanks to the ILAs. *probe0* is the output from the DUT, while *probe1* is the reference output from the ROM. It is clear that both of them are identical at any time, which proves that our design is working correctly and the synchronization scheme we used is reliable.

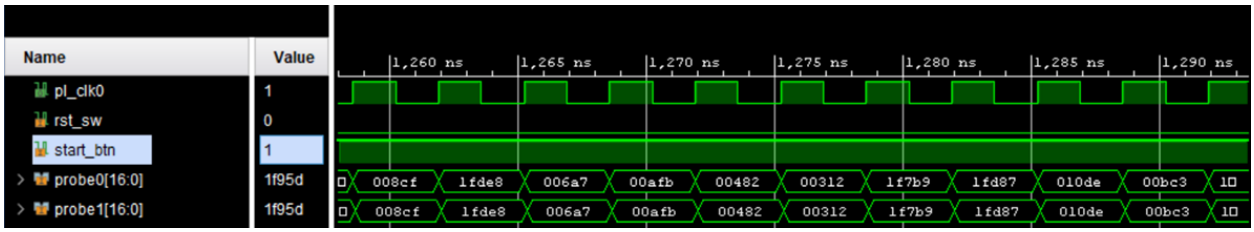


*Figure 13. FPGA Prototyping Results*